Report on Mini Project

Subject: Blockchain Technology

AY: 2026–27

Crowd Fund: Decentralized Crowdfunding dApp

Sarthak Agrawal: 2303002

Ahmed Ansari: 2303009

Sunil Bhagat: 2303020

Yash Bhardwaj: 2303021

Department of Computer Engineering

Thadomal Shahani Engineering College

CHAPTER 1: INTRODUCTION

1.1 Project Overview

Crowd Fund is a decentralized crowdfunding application (dApp) built on the Ethereum blockchain. It uses Solidity smart contracts to enforce campaign rules, manage contributions, hold funds in escrow, and control withdrawals and refunds. A web interface built with Next.js 13, React 18, Ethers.js v6, and Tailwind CSS allows users to create campaigns, contribute Ether, and inspect campaign status.

Unlike a conventional platform where an intermediary controls the ledger and custody of funds, Crowd Fund places its core financial rules in a smart contract. Contributions are held by the contract, campaign state is evaluated deterministically, and eligible refunds are initiated by contributors.

1.2 Problem Statement and Motivation

Traditional crowdfunding platforms depend on centralized operators to hold funds, maintain records, enforce campaign conditions, and process refunds. This creates custodial risk, information asymmetry, possible administrative suspension, intermediary charges, and uncertainty about refunds when a campaign does not reach its target.

- Custodial risk: funds are held by an intermediary rather than governed by transparent on-chain rules.
- Economic inefficiency: platform and payment-processing fees can reduce the capital available to a project.
- Accounting opacity: contributors may have to rely on the platform's own displayed figures.
- Refund uncertainty: failed-campaign refunds can depend on platform policies and manual processing.
- Geographical access barriers: conventional payment systems may not serve every contributor or creator.

1.3 Project Objectives

- Implement a transparent, smart-contract-based crowdfunding workflow.
- Keep contributed Ether in contract escrow until the campaign reaches its funding target.
- Prevent further donations after the target is reached.
- Return excess Ether when a final contribution exceeds the remaining target.
- Provide a controlled, one-time deadline extension during a seven-day decision window.
- Allow contributors to claim refunds when a campaign fails to reach its target and becomes refundable.
- Reduce common smart-contract risks through access control, reentrancy protection, and Checks-Effects-Interactions.

1.4 Scope and Boundaries

The current implementation accepts native Ether and is configured for a local Hardhat network with chain ID 31337. It demonstrates campaign creation, contributions, excess refunds, deadline handling, creator withdrawals, contributor refund claims, and a browser-based interface. It does not guarantee that a creator will deliver a real-world product after withdrawing successfully raised funds; that remains an off-chain risk.

1.5 Core Features

- Campaign creation with title, description, image URL, target amount, and future deadline.
- Wallet connection and transaction signing through MetaMask.
- Campaign lifecycle states: Active, GoalReached, DecisionPending, Refundable, and Completed.
- Exact wei-based accounting and automatic excess contribution return.
- Partial or full creator withdrawals after goal achievement.
- Individual pull-payment refund claims for eligible contributors.

CHAPTER 2: BLOCKCHAIN TECHNOLOGIES AND SUPPORTING STACK

2.1 Ethereum and the Ethereum Virtual Machine

Ethereum is a decentralized blockchain that supports smart contracts. The Ethereum Virtual Machine (EVM) executes compiled contract bytecode deterministically. Nodes process transactions and derive the same state transitions. EVM operations consume gas, which measures computational work and helps limit resource-intensive execution.

Ethereum uses two main account types. Externally Owned Accounts (EOAs) are controlled by private keys and can initiate signed transactions. Contract accounts contain bytecode and execute their programmed logic when called. The deployed CrowdFunding contract stores campaign state and enforces the platform's financial rules.

2.2 Solidity 0.8.19

Solidity is a statically typed, contract-oriented language for EVM-compatible smart contracts. The project uses Solidity 0.8.19 with the Paris EVM target. Arithmetic overflow and underflow checks are built into Solidity 0.8.x. Custom modifiers implement owner-only access and a mutex-style non-reentrancy guard. Ether transfers use low-level call with return-value verification.

2.3 Hardhat

Hardhat is the development environment used to compile, test, debug, and deploy the contract. Its local node exposes JSON-RPC at http://127.0.0.1:8545 and uses chain ID 31337. The local network provides funded test accounts and supports controlled timestamp manipulation for testing campaign deadlines, the seven-day decision period, and the maximum extension.

2.4 Ethers.js v6

Ethers.js connects the frontend and server-side API routes to Ethereum. BrowserProvider wraps MetaMask's injected provider; JsonRpcProvider supports server-side read requests; and Signer represents an account able to authorize transactions. The application uses parseEther and formatEther to convert between readable Ether values and exact wei amounts, with JavaScript BigInt preserving integer precision.

2.5 Next.js, React, Tailwind CSS, and MetaMask

- Next.js 13.5 App Router organizes pages and API route handlers.
- React 18 provides interactive client-side components and state management.
- Tailwind CSS supplies utility-based styling for the dashboard and modals.
- MetaMask manages the user's wallet connection, account selection, network context, and transaction signatures.
- The EIP-1193 provider interface supports communication with the injected wallet.

2.6 Technology Selection Rationale

The stack keeps smart-contract development, automated testing, and frontend integration within a JavaScript/Node.js toolchain while retaining Solidity for on-chain logic. Next.js API routes allow campaign data to be read without requiring every visitor to connect a wallet. Ethers.js v6 provides native BigInt handling, and Hardhat provides local EVM testing and time-control helpers.

2.7 Main Technology Stack

- Smart contract: Solidity 0.8.19.
- Blockchain development and local execution: Hardhat 2.18.0, local EVM, chain ID 31337.
- Web3 library: Ethers.js v6.7.1.
- Frontend: Next.js 13.5.4, React 18, Tailwind CSS.
- Wallet: MetaMask.
- Testing: Mocha, Chai, and Hardhat Network Helpers.

CHAPTER 3: SYSTEM ARCHITECTURE AND SMART CONTRACT LOGIC

3.1 High-Level System Architecture

The application uses a decoupled, multi-tier architecture. The browser contains the interface and wallet context. Next.js API routes retrieve read-only campaign data through a JSON-RPC provider. MetaMask signs transactions from the user, and the deployed CrowdFunding contract performs the authoritative state changes and escrow accounting.

USER / BROWSER

React + Next.js UI ↔ EthersContext ↔ MetaMask

↓ Read requests ↓ Signed transactions

Next.js API Routes → JsonRpcProvider → Local Hardhat EVM

↓

CrowdFunding.sol → Campaign State + Escrow + Events

The API routes /api/campaigns and /api/total support read operations. Before decoding contract data, the server checks that bytecode exists at the configured address. This handles the case where the local Hardhat node has restarted and the contract has not yet been redeployed.

3.2 Main Components

- Navbar and campaign cards: provide navigation, campaign summaries, progress, state labels, and available actions.
- EthersContext: manages provider detection, wallet synchronization, network validation, and account/chain change events.
- Create Campaign interface: collects campaign details and submits createCampaign after wallet confirmation.
- Withdraw modal: validates a creator's requested partial or full withdrawal.
- Extend modal: lets the creator request one deadline extension within the permitted decision period.
- Alert modal: confirms manual campaign closure.
- Next.js API routes: retrieve and serialize read-only campaign metrics for the interface.
- CrowdFunding.sol: enforces campaign state, access control, contribution accounting, refunds, and withdrawals.

3.3 Campaign Data Structures

The Donation structure records the contributor address and accepted amount in wei. The Campaign structure stores the owner, title, description, image URL, target, collected amount, withdrawn amount, deadline, donation records, goal status, manual-closure flag, extension count, and refunded amount.

The contract also maintains campaignCount, totalCollected, a mapping from campaign IDs to campaigns, and a nested contributions mapping keyed by campaign ID and contributor address. The nested mapping supports direct refund-amount lookup without iterating over every donation.

3.4 Campaign Lifecycle State Machine

- Active (0): the deadline is in the future, the target is unmet, and the campaign has not been manually closed.
- GoalReached (1): the collected amount has reached the target; additional contributions are blocked.
- DecisionPending (2): the original deadline has passed without reaching the target, no extension has been used, and the seven-day decision window remains open.
- Refundable (3): the target remains unmet and the decision period has expired, an extended deadline has expired, or the creator has manually closed the campaign.
- Completed (4): the goal was reached and the creator has withdrawn all eligible collected funds.

The getCampaignState(uint256 _id) view function resolves state dynamically from the campaign's target, collected and withdrawn amounts, closure flag, deadline, extension count, and current block timestamp.

3.5 Core Smart Contract Functions

Campaign creation — createCampaign

The function validates that the deadline is in the future and the funding target is greater than zero. It initializes a Campaign record, assigns msg.sender as owner, increments campaignCount, and emits CampaignCreated.

Contribution — donate

The function rejects zero-value contributions and campaigns that are not Active. It computes the remaining amount required to meet the target. If msg.value exceeds that amount, only the amount needed to reach the target is accepted and the surplus is returned to the contributor. Accounting is maintained in wei.

Deadline extension — requestDeadlineExtension

If the goal is unmet at the original deadline, the creator may request one extension during the seven-day decision period. The extension must be within the maximum permitted duration of 30 days. The contract prevents a second extension and rejects an extension deadline that is in the past.

Refund — claimRefund

When a campaign is Refundable, a contributor can initiate their own refund. The contract checks the caller's contribution, clears the recorded contribution before transferring Ether, updates the refunded amount, and verifies transfer success. This pull-payment pattern avoids a single failing recipient blocking refunds for all other contributors.

Creator withdrawal — withdraw

Only the campaign owner can withdraw eligible funds, and withdrawal is permitted only after the goal is reached. The contract tracks cumulative withdrawals and prevents the creator from withdrawing more than the remaining eligible amount.

3.6 Financial Accounting

All monetary values are stored as unsigned integer wei amounts. One Ether equals 10^18 wei. This avoids floating-point rounding in contract accounting. The frontend uses Ethers.js conversion helpers to display values in Ether while preserving exact integer values for transactions.

3.7 Transaction Workflow

1. The user fills out a campaign or contribution form in the web interface.
2. The frontend converts Ether values to wei and dates to Unix timestamps where required.
3. MetaMask asks the user to review and sign the transaction.
4. The signed transaction is sent to the local EVM through JSON-RPC.
5. The smart contract validates the request and updates state or reverts with an error.
6. The frontend refreshes the relevant information and displays a status notification.

CHAPTER 4: IMPLEMENTATION, TESTING AND RESULTS

4.1 Development Environment

The local implementation uses Node.js 18.x or 20.x, npm 9.x or later, Git, a modern browser, and the MetaMask extension. The smart-contract dependencies are installed in the smart-contract directory, the contract is compiled with Hardhat, and the test suite is run before local deployment.

4.2 Local Execution Procedure

1. Install the project dependencies in the smart-contract directory using npm install.
2. Compile the contract using npx hardhat compile.
3. Run the automated suite using npx hardhat test.
4. Start the local blockchain using npx hardhat node.
5. Deploy the contract in a second terminal using npx hardhat run scripts/deploy.js --network localhost.
6. Configure client/.env.local with the deployed contract address and local RPC URL.
7. Install client dependencies, run npm run dev, and open http://localhost:3000.
8. Configure MetaMask for Hardhat Localhost with RPC URL http://127.0.0.1:8545, chain ID 31337, and currency symbol ETH.

4.3 Implemented User Interface

The screenshots supplied with the project show the Crowd Fund landing page, campaign listing, campaign details, campaign creation, account page, wallet transaction confirmations, and creator withdrawal interface. The interface uses a dark theme with green action controls; these colours are part of the supplied application screenshots.

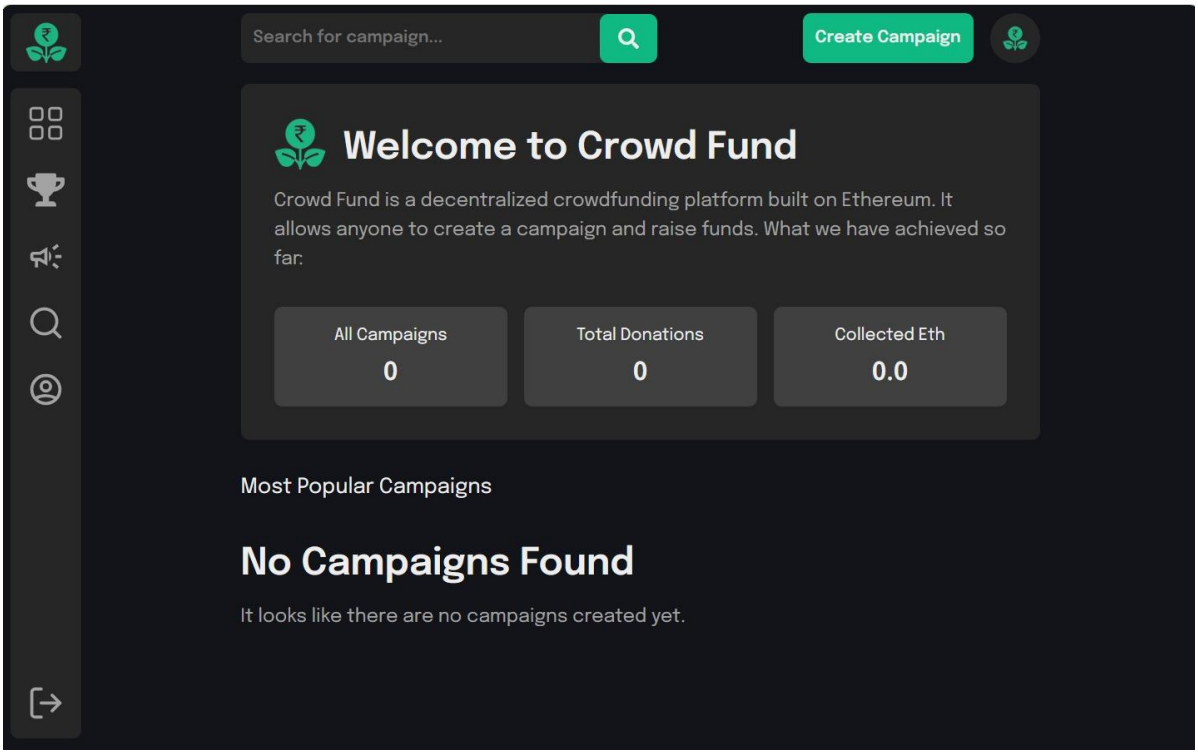


Figure 4.1: Crowd Fund landing page and campaign summary.

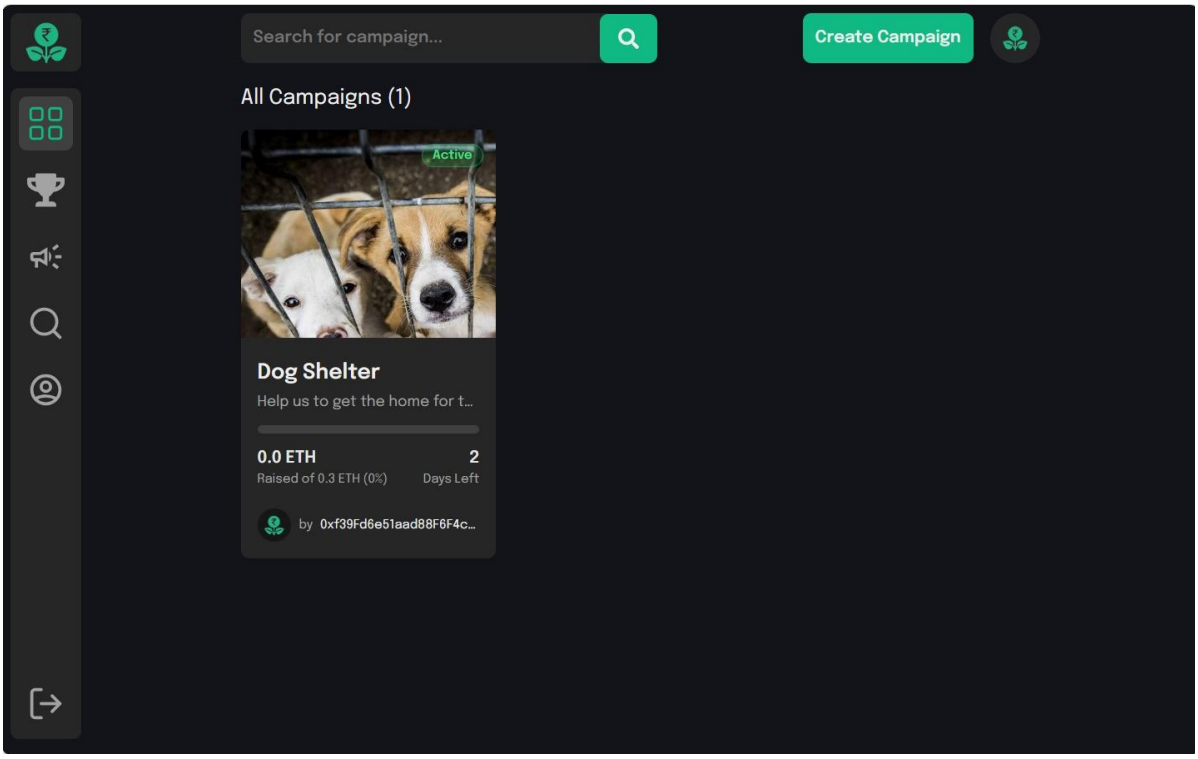


Figure 4.2: Campaign listing page displaying the available campaign.

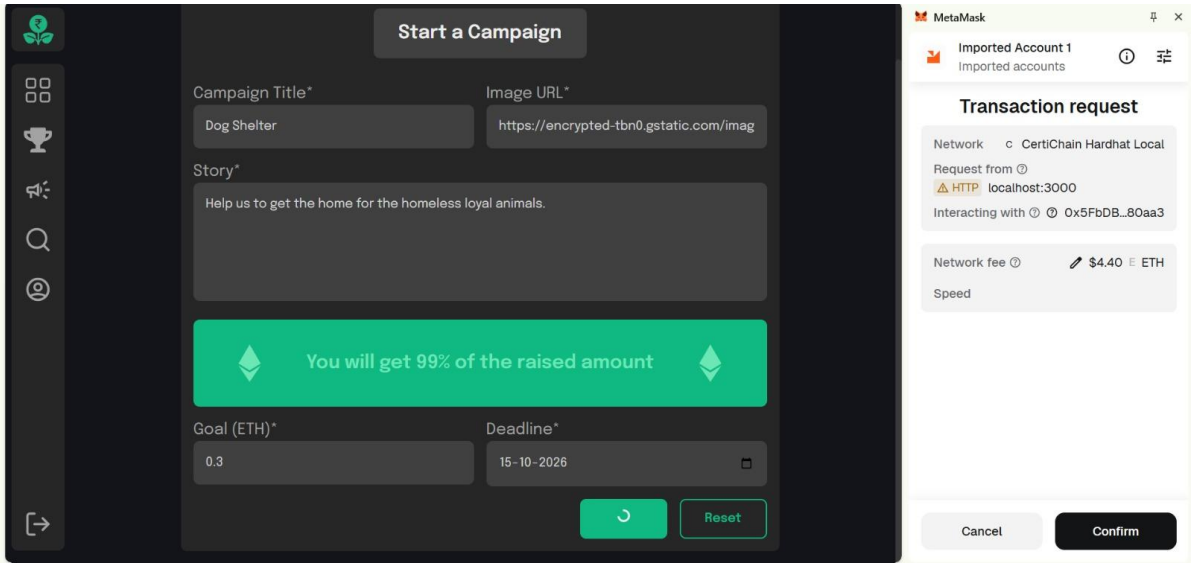


Figure 4.3: Create Campaign form with title, image URL, story, target and deadline fields.

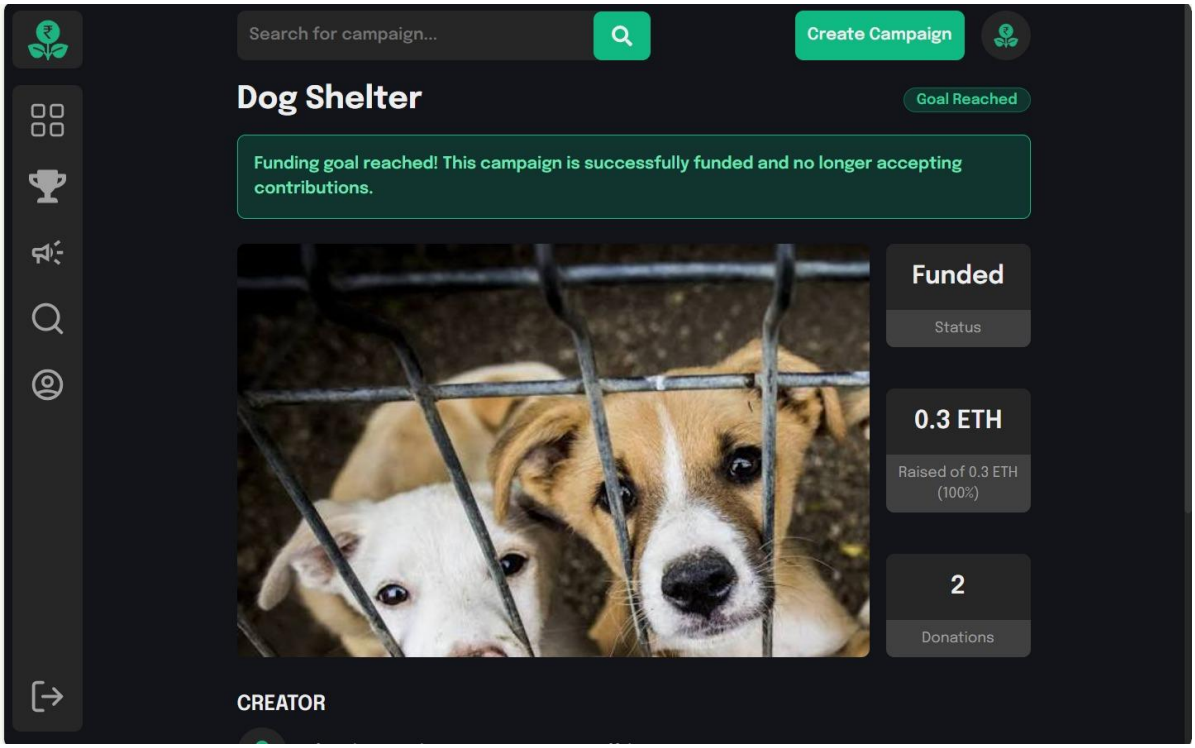


Figure 4.4: Campaign details page showing the funding status and contribution information.

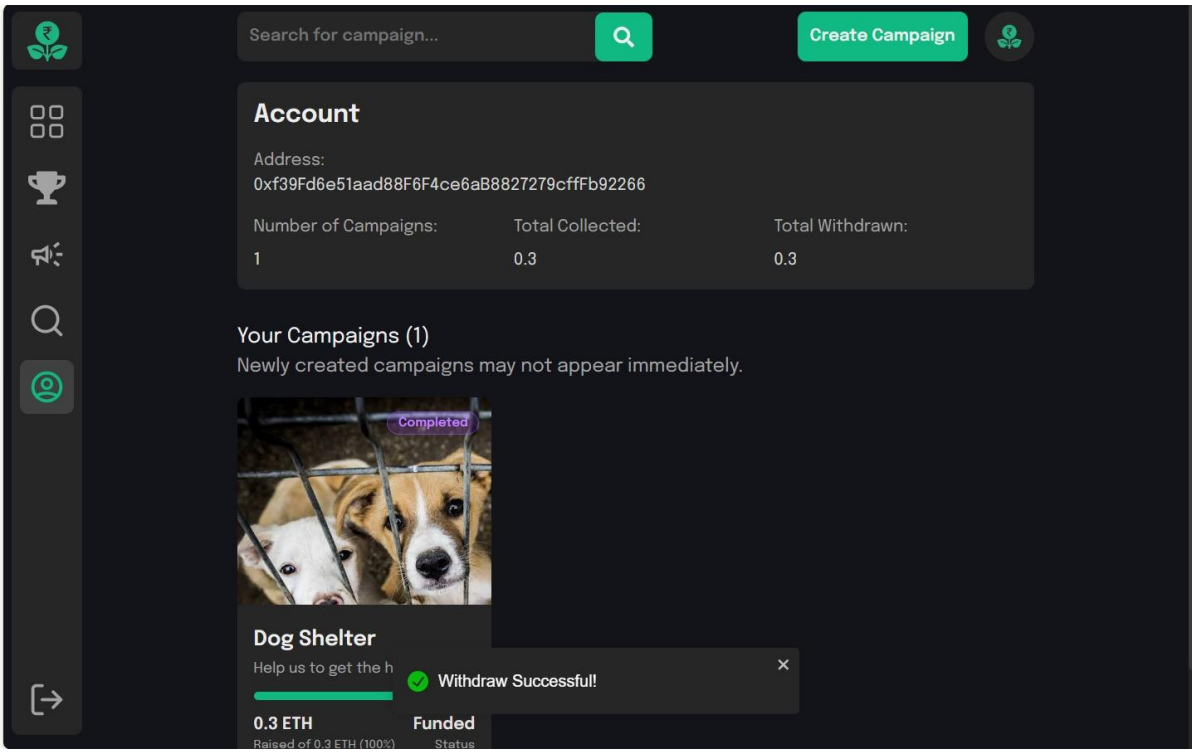


Figure 4.5: Account page showing campaign ownership and account-level statistics.

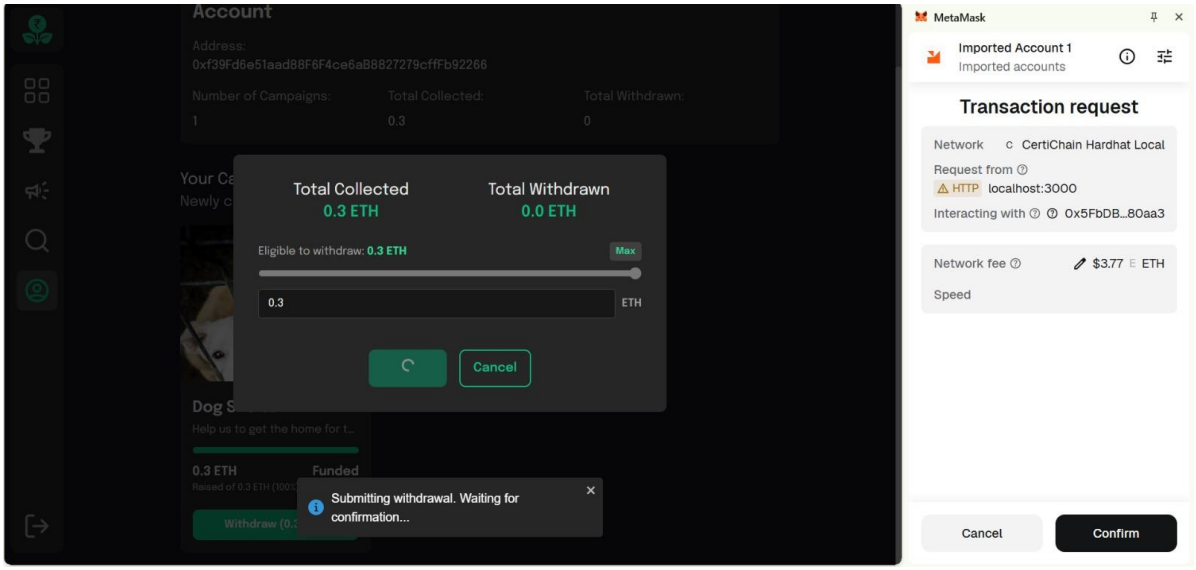


Figure 4.6: MetaMask transaction confirmation displayed during a campaign action.

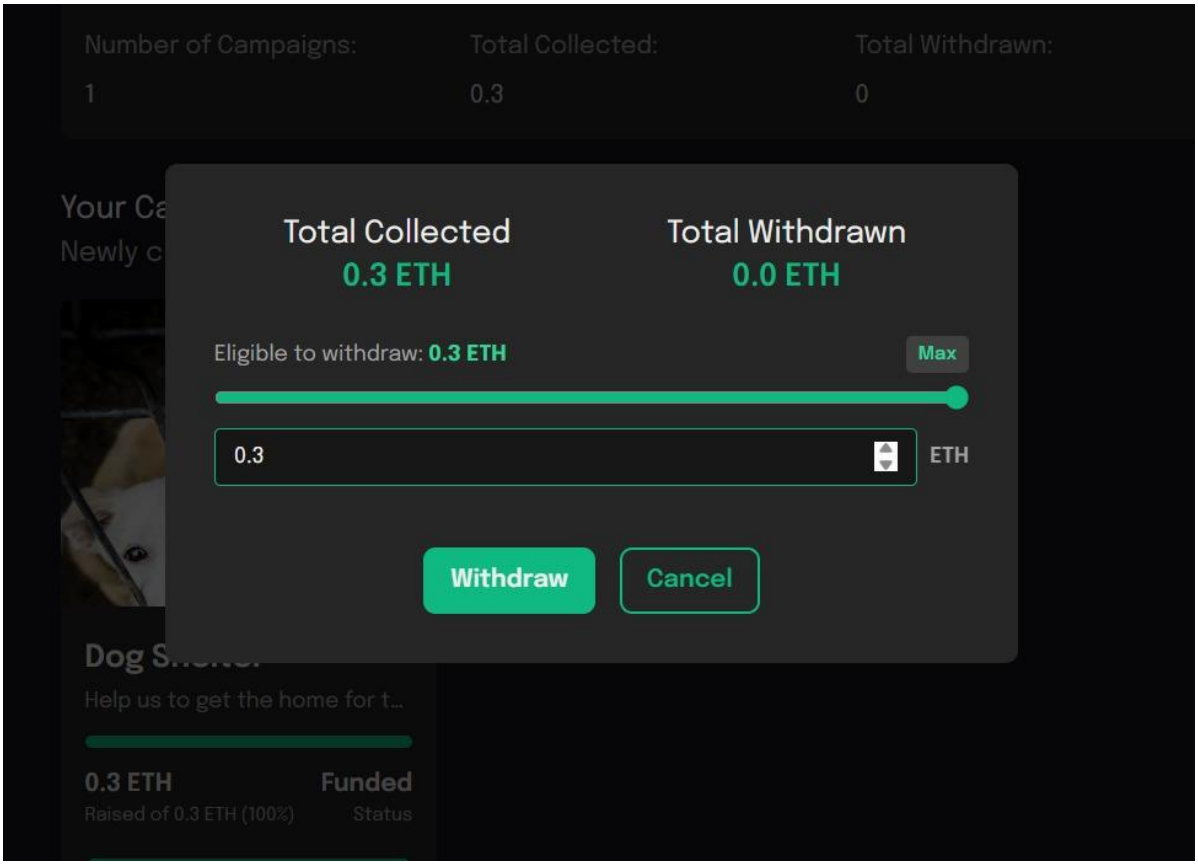


Figure 4.7: Withdrawal modal showing collected funds and the amount selected for withdrawal.

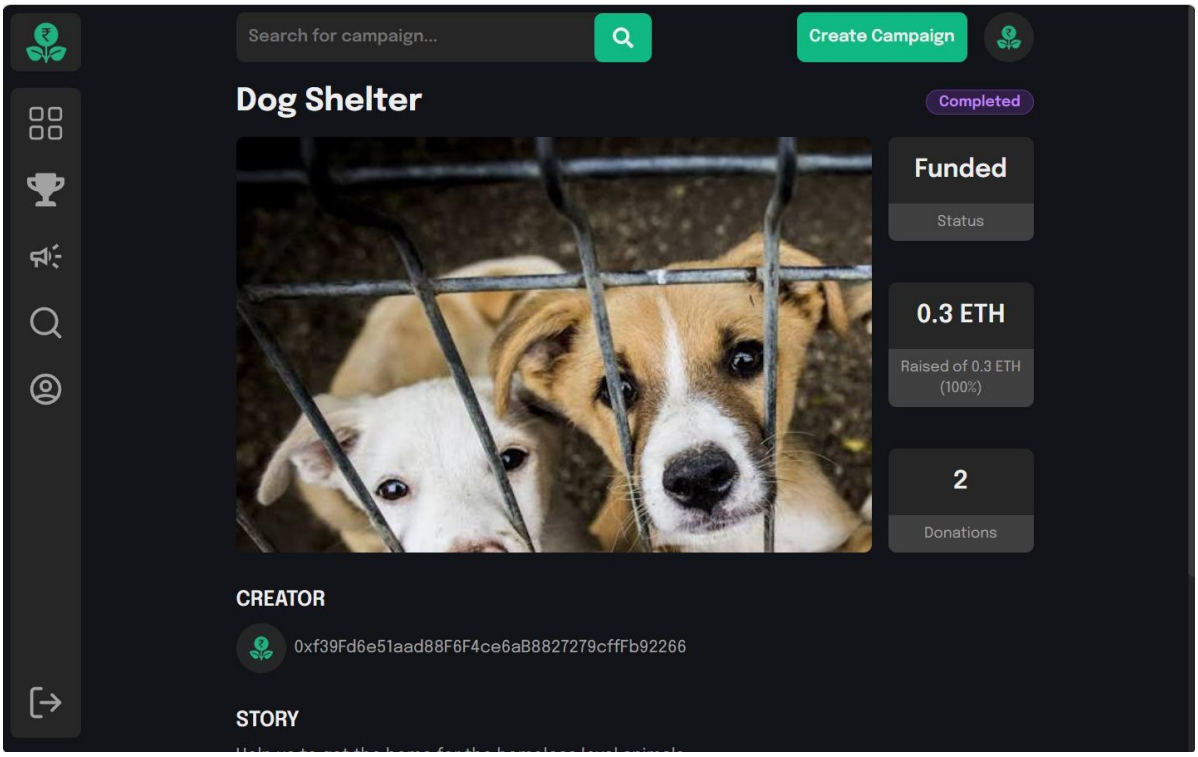


Figure 4.8: Campaign details after the campaign lifecycle reaches a completed state.

4.4 Automated Testing

The supplied test report records 29 passing tests. The suite covers campaign creation, invalid deadlines and targets, goal achievement, overfunding, deadline expiry, the decision period, one-time extensions, refunds, creator withdrawal, multiple contributors, and compatibility getters.

- Campaign creation: valid initial state; deadline must be in the future; target must be positive.
- Goal and overfunding: exact contribution reaches the goal; excess Ether is returned; donations stop after goal achievement.
- Deadline and decision window: expired campaigns reject donations; DecisionPending is entered when applicable; refunds are blocked during the decision window.
- Extension: valid one-time extension; contributions can resume; non-owners, excessive extensions, past deadlines, and second extensions are rejected.
- Refunds: refundable state is reached correctly; eligible contributors receive refunds; double claims and non-contributor claims are rejected.
- Withdrawals: failed-goal withdrawal is rejected; owners can withdraw after goal achievement; non-owner and zero-value withdrawals are rejected; partial withdrawal limits are enforced.
- Getters: campaign and donation list getters return the expected records.

Recorded test result: 29 passing (16 seconds). This result is based on the supplied implementation and testing document.

4.5 Result Analysis

The project demonstrates the core crowdfunding lifecycle through a local EVM: campaigns can be created, users can contribute Ether, excess amounts are handled, owners can withdraw eligible funds, and contributors can claim refunds when the contract state permits it. The automated tests exercise both successful and rejected transactions, including access-control and lifecycle boundaries.

4.6 Troubleshooting Notes

- If the local Hardhat node is restarted, redeploy the contract and update the client environment address as needed.
- A MetaMask nonce mismatch can occur if the local node is reset while the wallet retains old transaction activity; reset the local account activity in MetaMask.
- If a read request returns empty bytecode because the contract is not deployed at the configured address, deploy the contract before using the dashboard.

CHAPTER 5: SECURITY, LIMITATIONS AND FUTURE SCOPE

5.1 Threat Model

Public smart contracts operate in adversarial environments. Contract bytecode is visible, and mistakes in state updates, access control, or external transfers can cause irreversible financial loss. Crowd Fund uses defensive patterns to reduce common attack risks.

5.2 Reentrancy and Checks-Effects-Interactions

The contract applies a custom nonReentrant mutex to sensitive functions. It also follows Checks-Effects-Interactions: conditions are validated first, internal accounting is updated next, and external transfers occur last. For example, claimRefund clears the contributor's recorded amount before transferring Ether, preventing a reentrant call from claiming the same recorded contribution again.

5.3 Pull Payments and Denial of Service

Automatically refunding every contributor in a single loop can fail if one recipient rejects Ether, and a large contributor list can exceed the block gas limit. The pull-payment approach lets each contributor initiate an individual claim, so a failing refund does not block other contributors from claiming theirs.

5.4 Arithmetic and Access Control

Solidity 0.8.19 automatically reverts on integer overflow and underflow. The contract uses uint256 wei values to preserve exact accounting. The onlyOwner check restricts creator-specific operations to the campaign owner, while refund claims require the campaign to be Refundable and the caller to have a positive recorded contribution.

5.5 Timestamp Considerations

Block timestamps may have limited validator discretion. The project uses campaign periods measured in days, including a seven-day decision window and a maximum 30-day extension, so a small timestamp drift is negligible for these intended campaign windows.

5.6 Technical Limitations and Trust Boundaries

- Delivery risk: the contract controls escrow and eligibility, but cannot force a creator to deliver a real-world product after funds have been withdrawn.
- Centralized media links: campaign images use ordinary HTTP/HTTPS URLs; a broken or removed URL can make an image unavailable without changing financial accounting.
- Local network configuration: the current setup uses Hardhat chain ID 31337. A public testnet or mainnet deployment requires suitable RPC endpoints, funded gas wallets, and contract verification.
- Native Ether only: the current campaign accounting does not support ERC-20 tokens.
- No milestone governance: after goal achievement, the creator may withdraw eligible funds without a contributor vote on project milestones.

5.7 Future Scope

- ERC-20 stablecoin support using IERC20 and SafeERC20, with token-specific decimal handling.

- Decentralized campaign metadata and image storage using IPFS or Arweave.
- Milestone-based escrow release with contributor voting and pro-rata refunds for unreleased funds.
- Deployment on Ethereum Layer 2 networks such as Arbitrum, Optimism, or Base to reduce transaction costs.
- The Graph subgraph to index campaign events and provide search, filtering, and pagination without iterating over large on-chain arrays.
- Public testnet deployment on Sepolia and contract verification.
- Additional data validation, stronger deployment configuration, and more comprehensive production monitoring.

5.8 Conclusion

Crowd Fund demonstrates a decentralized crowdfunding workflow in which smart-contract rules govern campaign state, Ether escrow, excess contributions, withdrawals, and refunds. The Next.js interface and Ethers.js integration provide a practical way to interact with the contract, while Hardhat tests validate the main lifecycle and security boundaries. The current version is a local academic prototype; public deployment, decentralized media storage, token support, and milestone governance remain future improvements.

APPENDIX A: BLOCKCHAIN AND WEB3 GLOSSARY

ABI (Application Binary Interface): A JSON representation used to encode function arguments and decode return values when interacting with EVM bytecode.

BigInt: A JavaScript primitive for representing arbitrarily large integers without floating-point precision loss.

Bytecode: Compiled low-level instructions executed by the Ethereum Virtual Machine.

CEI (Checks-Effects-Interactions): A design pattern that validates conditions, updates internal state, and then performs external calls.

dApp: A decentralized application whose core business logic or settlement runs on a peer-to-peer blockchain network.

EIP-1193: A standard provider interface used by browser wallets to expose Ethereum connectivity.

EOA: An Ethereum account controlled by a private key.

EVM: The runtime environment that executes Ethereum smart-contract bytecode.

Gas: A unit measuring the computational effort required to execute an EVM operation.

Hardhat: A development environment for compiling, testing, debugging, and deploying smart contracts.

Immutability: The property that deployed contract bytecode and committed blockchain records cannot ordinarily be edited or deleted.

Mutex: A locking mechanism used to prevent concurrent or reentrant execution of protected code.

Nonce: An incrementing transaction counter for an EOA that helps prevent replay and sequence conflicts.

Pull payments: A payment model in which a recipient initiates their own withdrawal or refund.

Reentrancy attack: An exploit in which an external call re-enters a contract before its state updates are safely completed.

Signer: An account abstraction that can sign messages and authorize transactions.

Solidity: A statically typed language used to write smart contracts for EVM-compatible chains.

Wei: The smallest denomination of Ether; 1 ETH equals 10^18 wei.